

Vault Graph 小白指南

把整个 Obsidian 笔记库画成一张“圆盘星系”：文件夹是扇区，笔记是星星，连接多的居中。

开源项目 [luke321/vault-graph](https://github.com/luke321/vault-graph)

MIT 协议

TypeScript + WebGL

零网络请求

学习日期 2026-10-02 · 本指南属于 reminder 总仓库拾遗队列 · 所有实验命令均在本机实测

一句话总结：Vault Graph = 一个把 Obsidian 笔记库渲染成“确定性圆盘图”的插件 + 离线 HTML 导出器
——同一份源码，两种挂载；同一座笔记库，永远画出同一张图。

第一部分 · 12 个疑问逐条击破（疑问即目录）

这是什么？和 Obsidian 自带 graph view 有何不同？

01

Vault Graph 是一个 Obsidian 社区插件（263 star, MIT），把整座笔记库渲染成**一张交互式圆盘图**：每个顶级文件夹占一个扇形（角度=内容占比），每篇笔记是一个点，**被链接得越多越靠圆心**，“叶子笔记”落在边缘。

它还有第二形态：一条命令把整座库导出成**单个自包含 HTML 文件**，发给任何人、任何设备（包括手机）双击就能离线打开——不装 Obsidian 也能看。

与自带 graph view 的三大区别：① 自带视图是**力导向布局**（每次打开都可能长得不一样），Vault Graph 是**确定性布局**（同库同图）；② 它把**文件夹结构编码进图形**（扇区=归档体系）；③ 自带视图没有日期时间轴回放、没有单文件 HTML 导出。

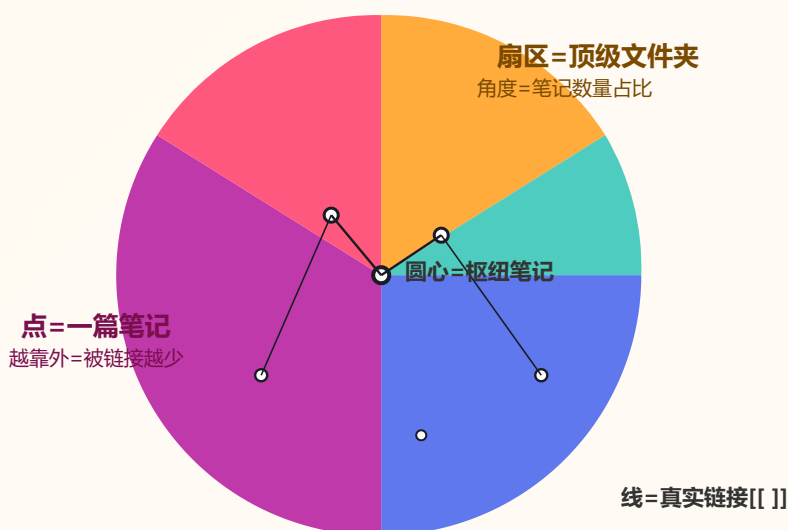


图1：圆盘图解剖——扇区是“归档体系”，边才是“真实结构”

项目 README 里有句话值得背下来：**"The wedges are the filing system; the edges are the actual structure."**（扇区代表你如何归档，边代表知识如何真正相连。）作者实测自己 vault 里只有约 9% 的链接留在同一个顶级文件夹内——知识天然跨文件夹流动的。

为什么“确定性布局”是最大卖点？力导向布局有什么毛病？

Obsidian 自带 graph、Gephi、d3-force 这类工具用**力导向布局**（force-directed）：把图想象成物理系统，点之间有斥力、边有弹簧拉力，跑模拟直到“力平衡”才停下来。毛病有三：① **结果不可复现**——初始位置随机、收敛过程混沌，同一座库每次打开长得都不一样；② 有点“抽卡”味道，运气不好就一团乱麻；③ 大图模拟收敛慢。

Vault Graph 反其道行之：位置由**纯几何规则算出来**——扇区角度由文件夹占比决定，环半径由链接数决定，角度内再按路径排序。**没有模拟、没有随机种子，同一座库永远画出同一张图。**这带来一个被低估的好处：图变得“可学习、可辨认”——你看一周后能记住“我的 MCP 知识都在左上那片蓝色扇区”，别人的圆盘截图你也能读出对方库的宏观结构。这正是知识库[[确定性脚本]]思想在可视化上的延伸：输入相同则输出相同，图形本身成为可引用的“事实”。



环半径：链接度排序 → 中心放"被引最多的枢纽笔记"，
外缘放"没人链接的叶子笔记"；子文件夹用父颜色的浅色调(tint)画一层。

图2：布局是两步算术——先按占比切蛋糕（扇区），再按链接度决定深浅（同心环）

细节加分项：`\_` 前缀文件夹（如 `\_templates`、`\_Archives`）被视为 **归档**——不占彩色槽、默认隐藏；日期型文件夹（`2026`、`2026-06`）有专门的第一层识别；子文件夹只画一层，最大的三个各配一个专属 tint。

Vault Graph 同时是一个 Obsidian 插件（`main.js`，跑在 Obsidian 窗口里）和一个独立 HTML 导出器（`vault-graph.html`，双击在浏览器打开）。作者没有写两套代码，而是 **把渲染页面做成同一个 HTML+JS，两个宿主各挂载一次**——插件把页面挂进 Obsidian 的视图容器，导出器把同一页面连数据打包成单文件。

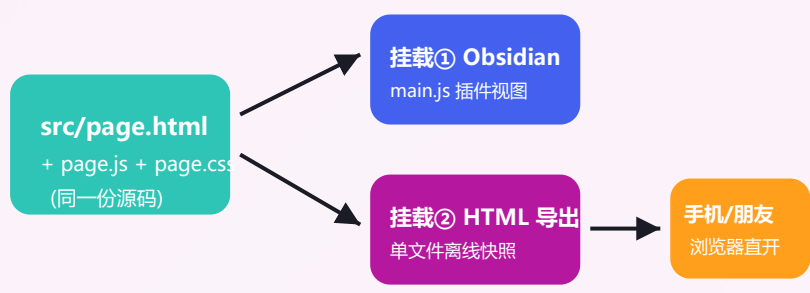


图3：one page, two mounts——一套渲染代码，两个宿主环境

这个模式对我们写自己的工具很有启发：**把"视图"和"宿主"解耦**，视图只吃一坨 JSON 数据，宿主（插件壳 / 导出器）负责喂不同的数据来源。构建时 esbuild 把页面和数据内联成一个自包含 HTML。

为什么整个项目能做到“零网络请求”？这有什么好处？

06

仓库里有个强制脚本 `scripts/check-network.mjs`——构建时扫描代码，**发现任何网络调用** (`fetch/XHR/WebSocket`) **直接构建失败**，连跳过开关都没有。另有 `check-pii.mjs` 拒绝把真实人名打进发布产物。

好处非常实在：① 导出的 HTML 是纯静态快照，**离线可用、断网可看、可放到手机**；② 隐私安全——你的笔记标题不会被发到任何地方，也没有遥测；③ 供应链安全——依赖极少（导出器只依赖 esbuild 一个包），没有“装个包顺手偷数据”的风险面。这是 [[零依赖编程]] 的工程化极端版：把“不联网”写成测试，靠 Gate 保证而不是靠自觉——正是知识库里 [[质检Gate与自我纠错循环]] 的思路。

WebGL 渲染引擎是从哪来的？为什么不用现成库？

07

几千个点+边的实时交互渲染，Canvas 2D 会吃力，所以作者用 WebGL。有趣的是引擎不是从零写的：

`src/engine/` 是从 **Sigma.js 3.0.2 (MIT 协议)** 移植来的精简版，移植后只保留圆盘图需要的那部分能力（点/边绘制、拾取、相机），署名保留在 `src/engine/NOTICE.md`。

为什么不直接依赖 sigma.js？因为要满足第 6 问的零依赖目标——移植进来意味着**没有运行时依赖、代码完全可控、产物更小**。MIT 协议允许这样做，条件是保留版权声明（他们做了）。这是一次教科书级的“依赖收敛”操作：先评估现成库，再用协议允许的方式把它“收编”成自己代码的一部分。

日期时间轴回放是怎么实现的？数据从哪来？

08

圆盘下方有一条**可拖动的日期带 (date ribbon)**，拖动时只显示“该日期之前创建”的笔记——就像看延时摄影一样回放笔记库的生长过程。旁边还有**日历热力图**，悬停可看某天写了几篇。

数据来源是笔记 frontmatter 里的 `created:` 字段，缺失时回退到 `date:`；**两者都没有的笔记始终显示**（不会被错误地排到时间轴尽头）。两个贴心细节：日记里常见的 `[[prev]]` | `[[next]]` 导航行默认会被算作链接（形成日记长链），CLI 加 `--no-nav` 可剥离；`--flat-months` 可把 `2026-08` 这类月份文件夹折叠进父级，避免日记把圆盘染成一大片。

CLI 导出 HTML 的完整流程？产物里到底有什么（隐私！）？

09

流程只要三步（下一部分实验实测）：**clone 仓库** → ``npm ci --omit=dev``（只装 esbuild 一个包） → ``node src/build-graph.mjs --vault 路径``，生成的 ``vault-graph.html`` 默认写进 vault 根目录（``--out`` 可改）。

⚠ 隐私红线：导出的 HTML 以明文包含每篇笔记的标题、路径、文件夹、标签、日期、字数和完整链接图（不含正文）。README 原话：“treat it as private by default”——把导出文件当私密文件对待，公开分享前想清楚。想展示可用项目自带的 `make-demo-vault.mjs`（虚构内容）或 `make-mirror-vault.mjs`（保留真实结构但内容全换）生成替代 vault。

全部 CLI 参数速查：`--vault` 指定库 / `--vault-name` 按注册名选库 / `--out` 输出位置 / `--ghosts` 显示指向不存在笔记的死链 / `--templates` 包含模板笔记（默认排除，因其占位链接是假边） / `--flat-months` 折叠月份 / `--no-nav` 剥离日记导航。

归档夹、死链、模板这些细节处理为什么重要？

10

可视化工具的成败往往不在主算法，而在**对“脏数据”的态度**。Vault Graph 的处理清单：

脏数据	默认处理	为什么
`_` 前缀文件夹	归档：深灰色、默认隐藏、不占色槽	归档不是活跃知识，画出来全是噪声
模板笔记	排除（ <code>`--templates`</code> 才包含）	模板里的占位链接是“假边”，会伪造连接度
指向不存在笔记的 [[死链]]	默认不画（ <code>`--ghosts`</code> 才显示）	死链是“计划中”的笔记，不是事实
日记导航行 prev/next	算链接（ <code>`--no-nav`</code> 剥离）	默认尊重真实链接，但给你关掉的自由
没有 frontmatter 日期的笔记	始终显示	缺数据≠不存在，不能把人排到时间轴外

这套清单和知识库 [[可逆派生状态]] 的精神一致：**图形上画的每一条边都应该有事实依据，“计划中/想象中”的东西要么明示要么不画。**

热力图和时间轴怎么配合看笔记增长？

11

两者是同一份日期数据的两种视图：**热力图**（calendar）回答“哪天高产”——GitHub 提交格子那种样式，悬停看具体数字；**时间轴**（date ribbon）回答“库是怎么长成现在这样的”——拖动滑块，圆盘上笔记点逐个亮起。实用场景：① 回顾某个月的知识重心（配合扇区颜色看“那个 month 主要在学什么”）；② 找出“孤岛期”（连续很久没新笔记的空档）；③ 检查 frontmatter 习惯——如果你的库时间轴一片空白，说明 ``created`` 字段没维护，这本身就是一次有价值的体检。

我怎么把它用在自己的 vault 上？分享/发布注意什么？

12

作为插件：Obsidian 设置 → 社区插件 → 浏览 → 搜 "Vault Graph" → 安装启用，点左侧功能区图标打开。适合日常探索。

作为导出器：Node 18+ 环境，三条命令（见实验）。适合季度回顾、给别人展示库结构、备份一份可视化快照。

分享守则：① 默认私密，分享前先想“这些标题能见人吗”；② 需要展示就用 demo/mirror vault 脚本生成替身；③ 下载官方发布版时可用 `gh attestation verify main.js --repo luke321/vault-graph` 验证构建来源证明（1.9.0 起提供）；④ 导出页面里浏览器偏好存在 localStorage，按 vault 名做键，禁站点数据也能正常用只是不记忆偏好。

第二部分 · 项目能力清单（它能给我什么）

5 个立刻能用的用途



- 知识库年度体检**：一条命令出图，看哪个文件夹膨胀成了巨无霸、哪些概念笔记是没人链接的孤岛，据此决定下一阶段该补链接还是该拆文件夹。
- 给别人介绍自己的知识体系**：导出单个 HTML 发过去，对方手机浏览器直接打开，不用装任何东西。
- 学习节奏回放**：时间轴+热力图回看自己三个月的知识增长，找出断更期。
- 找枢纽笔记**：圆心的笔记就是你的 MOC (Map of Content)，如果圆心是空的，说明该写总览笔记了。
- 学它的工程**：确定性布局、零网络 Gate、一份源码两种挂载、Sigma.js 移植收编——每个都是能搬走的工程模式。

第三部分 · 动手实验：给本仓库的知识库画一张圆盘

实验目标：用官方导出器，把 F:\reminder\vault（本仓库真实的 Obsidian 知识库，66+ 个概念笔记）只读渲染成单文件 HTML，并亲眼看到圆盘图。

实验步骤 1/5 • 拉代码并安装（唯一依赖 esbuild）

怎么操作：

```
git clone https://github.com/luke321/vault-graph repo
cd repo
npm ci --omit=dev
```

目的：拿到导出器源码；`--omit=dev` 只装生产依赖（仅 esbuild 一个包），验证它“依赖极少”的说法。

结果与验证：

```
PS repo> npm ci --omit=dev
added 1 package in 9s          # node_modules 里只有：
ls node_modules → esbuild  @esbuild/win32-x64
```

✅ **预期结果：**clone 成功且 node_modules 里只有 esbuild 相关包，几十秒装完。

实验步骤 2/5 • 对真实 vault 生成圆盘 HTML

怎么操作：

```
node src/build-graph.mjs --vault "F:\reminder\vault" --out "F:\reminder\vault-graph\exercise\vault-graph.html"
```

目的：用官方导出器渲染本仓库真实知识库（只读，不改动 vault 任何文件）。

结果与验证：

```
PS repo> node src/build-graph.mjs --vault "F:\reminder\vault" --out "...exercise\vault-graph.html"
vault-graph: 123 notes, 516 links, 2 orphans, 2 unresolved link(s)
dated: 0 from frontmatter, 0 from the filename, 123 from the file stamp, none undated
wrote ...\vault-graph.html (738 KB)
```

实测解读：本仓库 vault 有 123 篇笔记、516 条链接；2 条 unresolved 是“计划中还没写”的笔记；日期全部来自文件时间戳（我们没写 created frontmatter——这本身就是一次体检结论：想让时间轴更准，该补 frontmatter）。

✅ **预期结果：**生成 vault-graph.html，日志显示解析出的笔记数与链接数。

实验步骤 3/5 • 亲眼看到圆盘图 (截图存证)

怎么操作： 双击打开 exercise\vault-graph.html (或命令行)：

```
msedge --headless --disable-gpu --screenshot="F:\reminder\vault-graph\exercise\disc.png" --window-size=1600,1000 "file:///F:/reminder/vault-graph/exercise/vault-graph.html"
```

目的： 验证产物真正可离线渲染，并留下视觉证据。

结果与验证：

```
365441 bytes written to file ...\\exercise\\disc.png # 1600x1000 截图
```

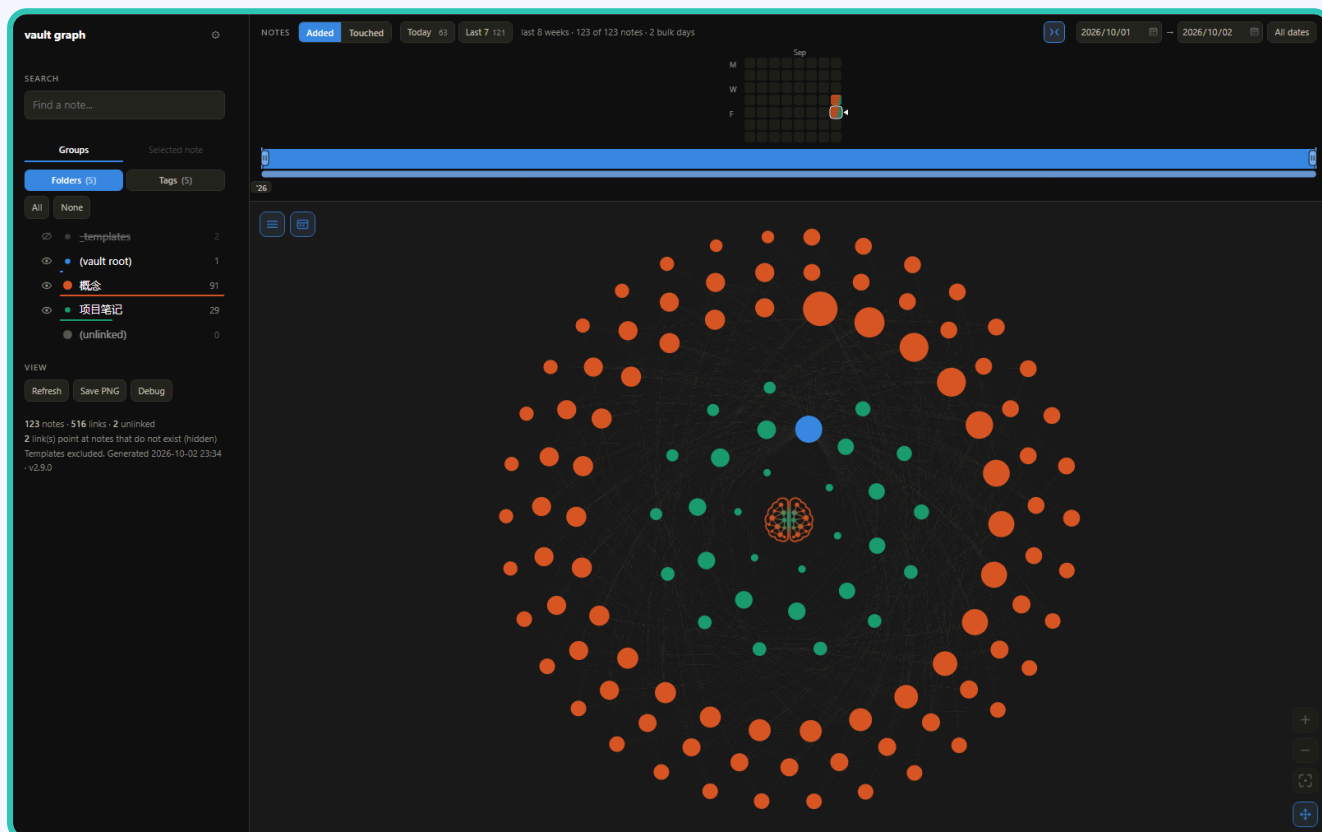


图4：本仓库 vault 真实渲染结果 (Edge 无头截图) —— 橙色大扇区=概念(91篇)，绿色=项目笔记(23篇)，蓝色=vault root(00-总览)，灰色=(unlinked) 孤儿笔记

✅ **预期结果：** 截图里出现彩色扇区圆盘，点在环上分布——实测可见“概念”扇区占了大半圆盘 (91/123)，枢纽笔记被圆盘中心的 logo 标出。

实验步骤 4/5 • 打开死链开关对比差异

怎么操作:

```
node src/build-graph.mjs --vault "F:\reminder\vault" --ghosts --out "F:\reminder\vault-graph\exercise\vault-graph-ghosts.html"
```

目的: 观察 `--ghosts` (显示指向不存在笔记的死链) 对图的影响, 体会"脏数据处理"开关的实际效果。

结果与验证:

```
vault-graph: 124 notes, 518 links, 2 orphans, 2 unresolved link(s)
wrote ...\\vault-graph-ghosts.html (739 KB) # 默认版是 123 notes / 516 links / 738KB
```

✅ **实测差异:** +1 个 ghost 笔记节点、+2 条边、文件 +229 字节——死链开关的效果肉眼可见地小 (我们的库只有 2 条死链), 但机制被清晰验证。

实验步骤 5/5 • 解剖产物: 确认"单文件自包含 + 零外链"

怎么操作:

```
grep -c "<script" exercise\vault-graph.html # 内联脚本数量
grep -c "http" exercise\vault-graph.html # 外部引用检查
ls -la exercise\vault-graph.html # 文件大小
```

目的: 验证 README 的两个承诺——单文件自包含、零网络请求。

结果与验证:

```
<script 标签数: 5 (全部内联)
src/href="http..." 外部资源引用: 0 个
(全文仅有的 http 字符串是 sigmajs.org 署名注释和 w3.org SVG 命名空间, 不是资源加载)
vault-graph.html          756,030 bytes
vault-graph-ghosts.html    756,259 bytes
```

✅ **实测结论:** 756KB 单文件, 样式脚本全内联、零外部资源引用, 断网可开——README 的两个承诺逐条兑现。

第四部分 • 一页速查

命令与参数速查 + 已知坑



```
# 生成（最常用）
node src/build-graph.mjs --vault "F:\你的库" --out "D:\输出.html"

# 常用开关
--ghosts      # 显示死链    --templates  # 包含模板笔记
--flat-months # 折叠月份夹  --no-nav      # 忽略日记导航

# 验证官方发布版签名
gh attestation verify main.js --repo luke321/vault-graph
```

已知坑：① `file://` 打开的页面禁止 fetch，页内 Refresh 按钮只是重置视图，更新数据必须重跑导出；② 导出器要 Node 18+；③ 导出 HTML 含笔记标题等明文元数据，分享前三思；④ 官方只实测了 Windows 冒烟测试，其他平台理论可用。

收尾一句话

Vault Graph 证明了：把“确定性、零依赖、隐私默认”当成硬约束来做可视化，产品反而更简单、更可信。这三个约束，值得写进我们自己每一个工具项目。